

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №1
по дисциплине «Информационные технологии»
Тема: «Парадигмы программирования.»

Студент гр. 5387

Степанов Леонид
Артемович

Преподаватель

Клюканов Виталий
Константинович

Санкт-Петербург
2026

СОДЕРЖАНИЕ

Цель работы	3
Задание.....	3
Основные теоретические положения.....	3
Выполнение работы.....	5
1. Проектирование иерархии классов изданий.....	5
2. Реализация пользовательских типизированных коллекций.....	7
Выводы	9
ПРИЛОЖЕНИЕ А	10

Цель работы

Изучение основ объектно-ориентированного программирования (ООП) на языке Python. Приобретение практических навыков проектирования иерархии классов, использования механизмов наследования, валидации входных параметров, а также переопределения стандартных магических методов и методов базовых коллекций.

Задание

Реализовать базовый класс `Edition` и производные классы `Book` и `Newspaper`. Во всех конструкторах обеспечить проверку типов и допустимых значений параметров; при некорректных данных выбрасывать `ValueError` с текстом 'Invalid value'. Для `Book` и `Newspaper` реализовать методы `__str__` и `__eq__` по заданным правилам. Реализовать классы `BookList` и `NewspaperList` на основе `list` с переопределением отдельных методов и функциями для вычисления характеристик.

Основные теоретические положения

Объектно-ориентированное программирование (ООП) представляет собой методологию разработки, при которой программа рассматривается как набор взаимодействующих объектов, каждый из которых является экземпляром определенного класса.

Основные концепции, применяемые в работе:

- Инкапсуляция и валидация: сокрытие внутренней реализации и защита состояния объекта от записи некорректных данных (например, отрицательной цены или пустого названия) путем проверок на этапе инициализации в конструкторе.
- Наследование: механизм, позволяющий описать новый класс на основе уже существующего (базового). Классы-наследники получают все свойства и методы родителя, но могут добавлять свои собственные. Вызов родительского конструктора в Python осуществляется функцией `super().__init__()`.

Добавлено примечание ([МОЭВМ1]): Для названий разделов «Цель работы», «Задание», «Основные теоретические положения», «Выполнение работы», «Выводы» - стиль «Заголовок 3» («ВКР_Заголовок 3»)

- Перегрузка операторов и магические методы: классы в Python могут изменять поведение встроенных операций. Метод `__str__` отвечает за удобочитаемое строковое представление объекта, а `__eq__` — за логику оператора сравнения `==`.

- Расширение встроенных типов: наследование от стандартных коллекций (например, `list`) позволяет создавать собственные структуры данных, которые ведут себя как обычные списки, но имеют дополнительные ограничения (например, типизация при добавлении) или уникальные методы обработки внутреннего содержимого.

Выполнение работы

1. Проектирование иерархии классов изданий

Коллекция, унаследованная от встроенного класса list.

Класс Edition

Базовый класс, описывающий абстрактное печатное издание.

- **Назначение:** Хранение общих атрибутов для всех видов изданий и первичная валидация данных.
- **Поля объекта:** name (название, строка), price (цена, целое число), age_limit (возрастное ограничение, целое число), style (стиль, строка 'c' или 'b').
- **Метод __init__(self, name: str, price: int, age_limit: int, style: str) -> None:**
 - Назначение: Конструктор класса.
 - Аргументы: Ожидает на вход название, цену, ограничение по возрасту и стиль.
 - Логика: Проверяет соответствие типов аргументов и их логическую корректность (строки не пустые, числа больше нуля, стиль строго равен 'c' или 'b'). При ошибке вызывает ValueError('Invalid value').

Класс Book

Производный класс от Edition, описывающий сущность «Книга».

- **Назначение:** Хранение специфичных для книг данных и определение их поведения.
- **Дополнительные поля:** author (автор, строка), hardcover (твердый переплет, логическое значение), pages (количество страниц, целое число).
- **Метод __init__(self, name: str, price: int, age_limit: int, style: str, author: str, hardcover: bool, pages: int) -> None:**

- *Логика:* Вызывает родительский конструктор через `super()`, затем валидирует собственные аргументы (число страниц > 0 , не пустой автор). Выбрасывает `ValueError` при несовпадении.
- **Метод `__str__(self) -> str:`**
 - *Назначение:* Возвращает форматированную строку с полным описанием книги.
- **Метод `__eq__(self, other: object) -> bool:`**
 - *Назначение:* Проверяет равенство двух книг. Возвращает `True`, если переданный объект является экземпляром `Book` и у них совпадают поля `name` и `author`.

Класс `Newspaper`

Производный класс от `Edition`, описывающий сущность «Газета».

- **Назначение:** Описание газетного издания.
- **Дополнительные поля:** `online_edition` (наличие онлайн версии, `bool`), `country` (страна, строка), `frequency` (периодичность в днях, целое число).
- **Метод `__init__(self, name: str, price: int, age_limit: int, style: str, online_edition: bool, country: str, frequency: int) -> None:`**
 - *Логика:* Вызывает конструктор `Edition`. Проводит валидацию собственных полей (периодичность > 0 , страна не пустая).
- **Метод `__str__(self) -> str:`**
 - *Назначение:* Возвращает форматированную строку с характеристиками газеты.
- **Метод `__eq__(self, other: object) -> bool:`**
 - *Назначение:* Возвращает `True`, если `other` является объектом `Newspaper`, и у обоих объектов равны поля `name` и `country`.

2. Реализация пользовательских типизированных коллекций

Коллекция, унаследованная от встроенного класса list.

Класс BookList

Коллекция, унаследованная от встроенного класса list.

- **Назначение:** Хранение списка книг с защитой от добавления объектов других типов.
- **Поля объекта:** name (название списка, строка).
- **Метод `__init__(self, name: str) -> None`:**
 - *Логика:* Инициализирует базовый список `super().__init__()` и сохраняет переданное имя в поле `self.name`.
- **Метод `append(self, p_object: object) -> None`:**
 - *Назначение:* Переопределенный метод добавления элемента в конец списка.
 - *Логика:* Проверяет с помощью `isinstance()`, является ли `p_object` книгой (Book). Если да — добавляет в список базовым методом `super().append()`, если нет — вызывает исключение `TypeError` с указанием неверного типа.
- **Метод `total_pages(self) -> int`:**
 - *Назначение:* Возвращает общее количество страниц во всех книгах списка (сумма значений полей `pages`).
- **Метод `print_count(self) -> None`:**
 - *Назначение:* Выводит в консоль текущее количество добавленных книг (с использованием функции `len()`).

Класс NewspaperList

Коллекция, унаследованная от встроенного класса list.

- **Назначение:** Хранение списка газет со специфичной логикой массового добавления элементов.
- **Поля объекта:** name (название списка, строка).

- **Метод `__init__(self, name: str) -> None:`**
 - *Логика:* Вызывает родительский конструктор и устанавливает имя списка.
- **Метод `extend(self, iterable: object) -> None:`**
 - *Назначение:* Переопределенный метод расширения списка элементами из итерируемого объекта.
 - *Аргументы:* `iterable` — объект, поддерживающий итерацию.
 - *Логика:* Проверяет наличие у `iterable` магического метода `__iter__`. Проходит по всем элементам и добавляет в коллекцию только те из них, которые являются экземплярами класса `Newspaper` (остальные игнорируются).
- **Метод `print_age(self) -> None:`**
 - *Назначение:* Находит и выводит в консоль минимальное значение возрастного ограничения (`age_limit`) среди всех газет в коллекции.
- **Метод `print_total_price(self) -> None:`**
 - *Назначение:* Вычисляет и выводит в консоль суммарную стоимость всех газет в списке.

Выводы

В результате выполнения лабораторной работы были успешно освоены ключевые принципы объектно-ориентированного подхода в языке Python. Была спроектирована и реализована иерархия классов предметной области (печатные издания) с применением механизма наследования.

Успешно реализована защита состояния объектов от записи некорректных данных за счет строгих проверок типов и значений в конструкторах классов с генерацией исключительных ситуаций. Продемонстрировано владение полиморфизмом посредством переопределения стандартных магических методов (`__str__`, `__eq__`).

Важным итогом работы стало создание специализированных контейнеров данных путем наследования от встроенного типа `list`. Переопределение методов `append` и `extend` позволило создать типизированные коллекции с безопасным добавлением элементов и уникальным бизнес-функционалом для подсчета общих статистических характеристик хранимых объектов. Поставленные цель и задачи работы выполнены в полном объеме.

ПРИЛОЖЕНИЕ А

Исходный код программы (Файл main.py)

```
class Edition:
    def __init__(self, name: str, price: int, age_limit: int, style: str):
        if not isinstance(name, str) or name == "":
            raise ValueError('Invalid value')
        if not isinstance(price, int) or price <= 0:
            raise ValueError('Invalid value')
        if not isinstance(age_limit, int) or age_limit <= 0:
            raise ValueError('Invalid value')
        if style not in ('c', 'b'):
            raise ValueError('Invalid value')

        self.name = name
        self.price = price
        self.age_limit = age_limit
        self.style = style

class Book(Edition):
    def __init__(self, name: str, price: int, age_limit: int, style: str,
                  author: str, hardcover: bool, pages: int):
        super().__init__(name, price, age_limit, style)

        if not isinstance(author, str) or author == "":
            raise ValueError('Invalid value')
        if not isinstance(hardcover, bool):
            raise ValueError('Invalid value')
        if not isinstance(pages, int) or pages <= 0:
```

```

        raise ValueError('Invalid value')

    self.author = author
    self.hardcover = hardcover
    self.pages = pages

    def __str__(self):
        return (f'Book: название {self.name}, цена {self.price}, "
                f"возрастное ограничение {self.age_limit}, стиль {self.style}, "
                f"автор {self.author}, твердый переплет {self.hardcover}, "
                f"количество страниц {self.pages}.")

    def __eq__(self, other):
        if isinstance(other, Book):
            return self.name == other.name and self.author == other.author
        return False

class Newspaper(Edition):
    def __init__(self, name: str, price: int, age_limit: int, style: str,
                  online_edition: bool, country: str, frequency: int):
        super().__init__(name, price, age_limit, style)

        if not isinstance(online_edition, bool):
            raise ValueError('Invalid value')
        if not isinstance(country, str) or country == "":
            raise ValueError('Invalid value')
        if not isinstance(frequency, int) or frequency <= 0:
            raise ValueError('Invalid value')

```

```

        self.online_edition = online_edition
        self.country = country
        self.frequency = frequency

    def __str__(self):
        return (f"Newspaper: название {self.name}, цена {self.price}, "
                f"возрастное ограничение {self.age_limit}, стиль {self.style}, "
                f"интернет издание {self.online_edition}, страна {self.country}, "
                f"периодичность {self.frequency}.")

    def __eq__(self, other):
        if isinstance(other, Newspaper):
            return self.name == other.name and self.country == other.country
        return False

class BookList(list):
    def __init__(self, name):
        super().__init__()
        self.name = name

    def append(self, p_object):
        if isinstance(p_object, Book):
            super().append(p_object)
        else:
            raise TypeError(f"Invalid type {type(p_object)}")

    def total_pages(self):

```

```
        return sum(book.pages for book in self)

def print_count(self):
    print(len(self))

class NewspaperList(list):
    def __init__(self, name):
        super().__init__()
        self.name = name

    def extend(self, iterable):
        if not hasattr(iterable, '__iter__'):
            raise TypeError
        for item in iterable:
            if isinstance(item, Newspaper):
                super().append(item)

    def print_age(self):
        if self:
            print(min(n.age_limit for n in self))

    def print_total_price(self):
        print(sum(n.price for n in self))
```